

Ein einfacher Parser für generalisierte Phrasenstrukturgrammatiken

Auf den folgenden Seiten wird der Aufbau eines Parsers für generalisierte Phrasenstrukturgrammatiken (GPSGs) beschrieben, der Grammatiken des in GKPS spezifizierten Formats verarbeitet. – Im einleitenden Abschnitt werden einige der Probleme skizziert, die sich für die Entwicklung eines Parsers aufgrund bestimmter systematischer und formaler Annahmen der GPSG stellen. Der zweite Teil diskutiert die für den Parser zentrale Annahme: daß das Prinzip der freien Merkmalinstantiierung durch ein Prinzip der „lexikalisch determinierten Instantiierung“ ersetzt werden kann. – Die sich daran anschließenden drei Abschnitte beschreiben die verschiedenen Teile des Programms: den Pre-Compiler, der aus mehreren Typen von Regeln einer GPSG kontextfreie Regeln erzeugt, den eigentlichen Parser und den Post-Filter, der die vom Parser erzeugten Strukturen auswertet.¹

1. Generalisierte Phrasenstrukturgrammatiken und kontextfreie Grammatiken

Da generalisierte Phrasenstrukturgrammatiken zu der Klasse der Unifikationsgrammatiken² gehören, die in einem systematischen Zusammenhang mit kontextfreien Grammatiken stehen, bildete folgende einfache Überlegung den Ausgangspunkt für die Entwicklung des hier beschriebenen Parsers:

Für kontextfreie Grammatiken gibt es effiziente Parsingalgorithmen. GPSGs sind spezielle kontextfreie Grammatiken. Also läßt sich das Problem, einen Parser für GPSGs zu entwickeln, darauf reduzieren, einen Parsingalgorithmus für kontextfreie Grammatiken so zu modifizieren, daß er den Charakteristika einer GPSG Rechnung trägt.

Ein Blick auf die formalen Differenzen zwischen kontextfreien Grammatiken und GPSGs läßt allerdings erkennen, daß einige tiefgreifende Änderungen erforderlich sind.

- (i) *Komplexe Symbole.* Wie in allen Unifikationsgrammatiken werden in einer GPSG zur Bezeichnung von syntaktischen und lexikalischen Kate-

1 Ich danke an dieser Stelle Herrn Hagen Langer für seine Anregungen und seine Unterstützung.

2 Zu dieser Klasse von Grammatiken zählen u.a. auch die *lexikalisch-funktionale Grammatik*, die *kategoriale Unifikationsgrammatik* und die *funktionale Unifikationsgrammatik*.

gorien statt einfacher Symbole wie 'S', 'NP', 'N' ... komplexe Symbole verwendet. Ein komplexes Symbol besitzt in einer GPSG keine hierarchische interne Struktur; es wird aus einer (endlichen) Menge von Merkmalspezifikationen gebildet. Jede Merkmalspezifikation ist ein geordnetes Paar, das aus einem Merkmal und einem Merkmalswert besteht.

- (ii) *Feature Specification Defaults und Feature Cooccurrence Restrictions.* Beide Typen von Regeln, die FSDs und die FCRs, beeinflussen die Bildung von komplexen Symbolen. Die FCRs schränken die Kombinierbarkeit von Merkmalspezifikationen innerhalb eines komplexen Symbols ein. Nicht jede Menge von Merkmalspezifikationen bezeichnet eine (mögliche) syntaktische Kategorie. Die FSDs dagegen legen Standardwerte für bestimmte Merkmale fest, wobei die Zuweisung solcher Standardwerte von dem Vorkommen anderer Merkmalspezifikationen abhängig sein kann. Was ihre logische Form betrifft, so werden FSDs und FCRs meistens als konditionale oder bikonditionale Aussagen notiert.

- (iii) *Immediate Dominance und Linear Precedence.* Phrasenstrukturregeln, wie man sie aus kontextfreien Grammatiken kennt, gibt es in einer GPSG nicht. Ihre Funktion übernehmen zwei Typen von Regeln: die ID-Regeln, die Dominanzbeziehungen zwischen Kategorien und Mengen³ von Kategorien determinieren, und die LP-Regeln, die Links-Rechts-Beziehungen zwischen den Kategorien festlegen, die die rechte Seite einer ID-Regel bilden.

- (iv) *Metaregeln.* Neben den ID- und den LP-Regeln gibt es einen weiteren wichtigen Regeltyp: die Metaregeln. Formal betrachtet, handelt es sich bei den Metaregeln um Funktionen, die aus lexikalischen ID-Regeln Mengen neuer lexikalischer ID-Regeln erzeugen.

Durch die Restriktion des Argumentbereichs einer Metaregel auf solche ID-Regeln, die nicht das Ergebnis ihrer Anwendung auf eine andere ID-Regel bilden (GKPS: 72, Definition 8), ist sichergestellt, daß die Menge der Regeln, die aus der Anwendung einer endlichen Menge von Metaregeln auf eine endliche Menge von lexikalischen ID-Regeln resultieren, selbst wieder endlich ist.

- (v) *Universal Feature Instantiation Principles.* Die drei Instantiierungsprinzipien (Foot-Feature Principle, Control-Agreement Principle und Head-Feature Convention) stellen die signifikanteste Abweichung vom Konzept der Phrasenstrukturgrammatik dar.

Anders als die FSDs und FCRs, die sich auf die interne Struktur der komplexen Symbole beziehen, ohne dabei kontextuelle Aspekte zu berücksichtigen, liegt ihre Funktion gerade darin, die Menge der mit einer

³ Genaugenommen handelt es sich nicht um Mengen, sondern um sog. „Multisets“, d.h. um Mengen, in denen ein Element mehrfach vorkommen kann.

ID-Regel kompatiblen Strukturen einzuschränken, indem sie, abhängig von dem „lokalen“ Kontext, das Vorliegen/Fehlen bestimmter Merkmalspezifikationen und Merkmalswerte in den Kategorien der ID-Regel fordern.

Bei der Entwicklung eines Parsers für GPSG lassen sich zwei grundsätzlich verschiedene Strategien verfolgen.

Wenn man sich eng an der Darstellung der Theorie generalisierter Phrasenstrukturgrammatiken in GKPS orientiert, dann liegt es nahe, den Parser so zu organisieren, daß bestimmte Mechanismen parallel operieren (s. Fig. 1): die ID-

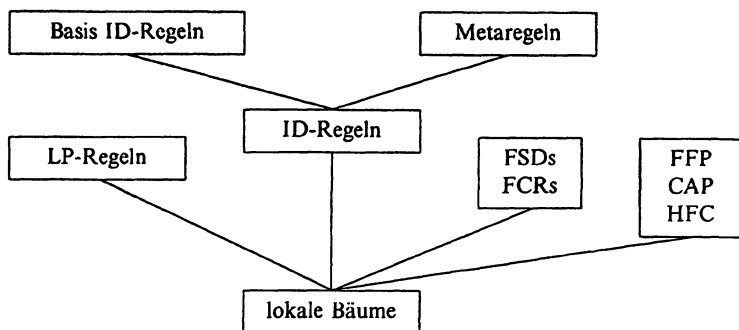


Fig. 1

Regeln, die LP-Regeln, die FSDs und FCRs und die universellen Merkmalsinstantiierungsprinzipien wirken zusammen, um die Menge der zulässigen lokalen Bäume festzulegen.⁴ Eines der Probleme, die sich bei dem Versuch einer *direkten Realisierung* dieses Konzeptes sofort stellen, besteht darin, die Interaktion der verschiedenen Regeltypen exakt zu spezifizieren. Dieses äußerst komplexe Problem wird in GKPS fast vollkommen ausgespart. Hinzukommt, daß der Parser die Regeln nicht als Wohlgeformtheitsbedingungen interpretieren kann, sondern sie als Generierungsvorschriften auffassen muß.

Ältere Arbeiten zur Theorie generalisierter Phrasenstrukturgrammatiken (Gazdar 1982), in denen die Unterscheidung von Metagrammatik und Objektgrammatik eine wichtige Rolle spielt, legen ein anderes Vorgehen, einen *indirekten Ansatz*, nahe: Soweit es geht, werden die verschiedenen Typen von Regeln einer GPSG nicht beim Parsen berücksichtigt, sie werden vielmehr dazu verwendet, eine Menge von kontextfreien Regeln mit komplexen Symbolen zu erzeugen, mit denen dann der Parser arbeitet.

4 In Shieber (1985) findet sich ein Algorithmus, der es ermöglicht, bestimmte Teile einer GPSG in den PATR-II Formalismus zu überführen.

Diese Lösung führt zur Aufgabe der Idee einer parallelen Organisation wichtiger Teile der Grammatik und berührt nicht das Problem der Interaktion der verschiedenen Regeltypen. Andererseits bietet sie die Möglichkeit, Parsingalgorithmen zu verwenden, deren formale Eigenschaften bekannt sind, und führt zu einem klaren Ableitungsbegriff, der dem kontextfreier Grammatiken sehr nahe kommt.

Der auf den folgenden Seiten beschriebene Parser basiert auf diesem zweiten Ansatz. Das Programm besteht aus insgesamt drei Teilen: dem *Pre-Compiler*, dem *Parser* und dem *Post-Filter* (s. Fig. 2).

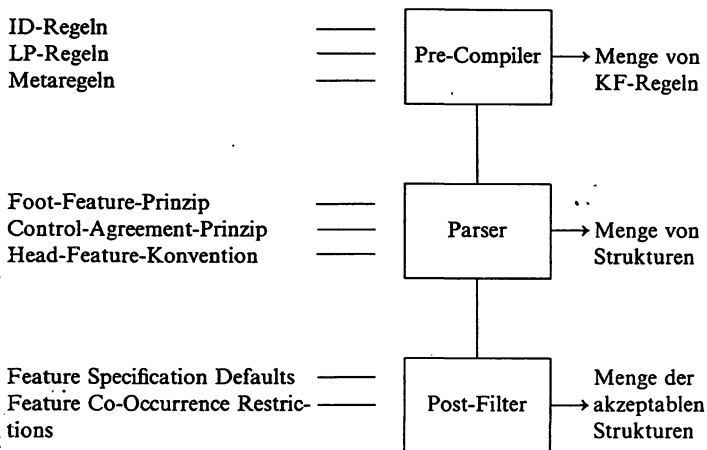


Fig. 2

Die Aufgabe des Pre-Compilers liegt darin, aus den ID-Regeln, den LP-Regeln und den Metaregeln eine (endliche) Menge von kontextfreien Regeln mit komplexen Symbolen zu erzeugen.

Der nach dem „active chart“-Prinzip arbeitende Parser, der diese Regeln verwendet, ist so konzipiert, daß er für einen Satz nur solche Strukturen erzeugt, die den Instantiierungsprinzipien genügen.

Der Post-Filter operiert auf den vom Parser gebildeten Strukturen: Er ergänzt Kategorien um weitere Merkmale und filtert die Strukturen aus, die Kategorien enthalten, die nicht den FCRs genügen.

Das Programm wurde in LISP geschrieben, genauer gesagt: in einem *Common Lisp*-Dialekt für MS-DOS Rechner. Das gesamte Programm umfaßt 37 KB, wobei 10 KB auf den Konverter, 22 KB auf das Analyseprogramm und 5 KB auf den Post-Filter entfallen. Die Verarbeitungszeit ist schon bei kleinen Grammatiken sehr unbefriedigend, was allerdings in Anbetracht der verwendeten

Rechner sowie der Tatsache, daß kein Compiler zur Verfügung stand, nicht überraschend ist.

2. „Freie“ oder lexikalisch determinierte Instantiierung von Merkmalen

Bevor ich in den nächsten drei Abschnitten auf die einzelnen Programmteile eingehe, halte ich es für erforderlich, die deutlichste Abweichung von der in GKPS entwickelten Theorie generalisierter Phrasenstrukturgrammatiken zu nennen. Die Annahme, auf der der Parser basiert und deren Akzeptanz über die des Parsers entscheidet, läßt sich wie folgt formulieren:

Es ist möglich, das Prinzip der freien Merkmalinstantiierung durch ein Prinzip der „lexikalisch determinierten Instantiierung“ zu ersetzen, ohne daß die Beschreibungsadäquatheit von GPSGs darunter leidet.

Der Parser basiert auf der Annahme, daß es keine „freie“ Instantiierung von Merkmalspezifikationen gibt, daß vielmehr jede Instantiierung durch bestimmte Faktoren determiniert ist. Eine entscheidende Rolle kommt dabei dem Lexikon zu: Die in ihm enthaltenen Informationen gelangen zunächst in die lexikalischen Kategorien und werden dann durch die Instantiierungsprinzipien über die erzeugten Strukturen verteilt. Die FSDs und FCRs verlieren so erheblich an Bedeutung. Die FSDs instantiieren, wo möglich, weitere Merkmale in Kategorien, und die FCRs testen, ob die erzeugten Strukturen akzeptabel sind (s. § 5).⁵

Unter Vernachlässigung der FSDs lassen sich drei Fälle unterscheiden, in denen Merkmale in Kategorien instantiiert werden dürfen. Wenn $K = \{k_1, \dots, k_n\}$ ($n \geq 1$) eine beliebige nicht-leere Kategorie ist und m eine Merkmalspezifikation, für die gilt $m \neq k_i$ ($1 \leq i \leq n$), dann ist m in K zu instantiieren gdw. eine der folgenden drei Bedingungen erfüllt ist:

- (i) K ist eine lexikalische Kategorie, und m ist eine Merkmalspezifikation, die in dem Lexikoneintrag des Wortes bzw. der Wortform enthalten ist, die in der vom Parser zu generierenden Strukturbeschreibung von K direkt dominiert wird. Anders ausgedrückt: In einer Ableitung α für einen Satz $S = w_1 \dots w_n$ ($n \geq 1$) wird eine Regel:

$$x \rightarrow y_1 \dots y_i K y_{i+1} \dots y_p$$
 angewendet, und w_i ($1 \leq i \leq n$) fällt unter die Kategorie K . M_{w_i} sei die Menge der im Lexikon für w_i angegebenen Merkmalspezifikationen, und es gilt: $m \in M_{w_i} - (K \cap M_{w_i})$ (*lexikalische Instantiierung*).
- (ii) K ist keine lexikalische Kategorie. Wenn in einer Ableitung α für einen

5 So einschneidend, wie dieser Eingriff zunächst scheint, ist er nicht: Zum einen ist der Spielraum für die freie Instantiierung von Merkmalen durch die verschiedenen Regeltypen stark eingeschränkt, zum anderen ist – abgesehen von Implementierungsproblemen – unklar, ob es eine linguistisch plausible Interpretation für diesen Mechanismus gibt.

beliebigen Satz S auf K eine Regel $r = K' \rightarrow y_1 \dots y_n$ angewandt wird, dann ist m in K zu instantiieren, wenn $m \in K' - (K \cap K')^6$ (*regelbedingte Instantiierung*).

- (iii) K ist eine beliebige Kategorie eines lokalen Baums b , und b erfüllt die drei Instantiierungsprinzipien nur dann, wenn m in K instantiiert wird (*prinzipienbedingte Instantiierung*).

Ein entscheidender Unterschied zwischen dem Ableitungsbegriff in kontextfreien Grammatiken und dem in GPSGs resultiert daraus, daß in einer GPSG G keine monadischen, sondern komplexe Symbole verwendet werden. Für eine beliebige kontextfreie Grammatik $G' = \langle V_N, V_T, S, R \rangle$ gilt: $w \Rightarrow w'$ gdw.

- (i) $w = \alpha x \beta$, $w' = \alpha y \beta$, mit $x \in V_N$ und $\alpha, \beta, y \in (V_N \cup V_T)^*$, und
 (ii) in R gibt es eine Regel der Form $x \rightarrow y$.

In G wäre die Forderung der Identität beider Kategorien zu restriktiv. Eine naheliegende Möglichkeit bestände darin zu fordern, daß beide Kategorien *unifizierbar* sein müssen. Ich habe mich bei der Entwicklung des Parsers für eine andere Lösung entschieden, die es erlaubt, die Expansion von Kategorien von zusätzlichen Restriktionen abhängig zu machen.

Der Parser testet zunächst, ob beide Kategorien *kompatibel* sind, und instantiiert dann in jeder der beiden Kategorien die Merkmale, die nicht in ihr, wohl aber in der anderen Kategorie enthalten sind. Dieses Verfahren könnte als *Unifikation durch Instantiierung* bezeichnet werden. Um eine korrekte Anwendung der Instantiierungsprinzipien zu garantieren, werden alle instantiierten Merkmale einer Kategorie unabhängig von den inhärenten Merkmalen der Kategorie verwaltet.

Definition 1: Kompatibilität

Wenn *Atom* die Menge aller atomwertigen Merkmale ist und $K = \{k_1, \dots, k_n\}$, $K' = \{k'_1, \dots, k'_m\}$ zwei beliebige Kategorien sind, dann sind K und K' *kompatibel* gdw.

- (i) $k(V) = K'(V)$;
 (ii) $K(N) = K'(N)$;
 (iii) $K(\text{BAR}) = K'(\text{BAR})$;
 (iv) für jedes Merkmal $m \in \text{Atom}$ gilt:
 wenn $m \in K \cap K'$, dann ist $K(m) = K'(m)$, und
 (v) für jedes andere Merkmal m gilt:
 wenn $m \in K \cap K'$, dann ist $K(m)$ kompatibel mit $K'(m)$.

Aus der Definition folgt, daß zwei Kategorien genau dann miteinander kompatibel sind, wenn in ihnen die Merkmale N , V und BAR dieselben Werte haben,

6 Da der Parser Strukturen zunächst als Folgen von lokalen Bäumen verwaltet, die erst abschließend in Strukturbäume transformiert werden, werden außerdem in K' alle Merkmale aus K instantiiert, die nicht in K' spezifiziert sind.

alle in beiden Kategorien spezifizierten atomwertigen Merkmale dieselben Werte haben und die Werte der in beiden Kategorien spezifizierten kategorienwertigen Merkmale miteinander kompatibel sind.

Der durch diese Definition festgelegte Kompatibilitätsbegriff ist weiter, als der in GKPS verwendete Begriff der *Extension*. Zwei Kategorien K , K' können miteinander kompatibel sein, ohne daß gilt: K ist eine Extension von K' bzw. K' ist eine Extension von K .⁷

Andererseits berücksichtigt er die besondere Funktion der Merkmale N , V , wie die Unterscheidung zwischen 'major' und 'minor categories' (Haupt- und Nebenkategorien) und ist insofern restriktiv genug, um unsinnige Resultate zu vermeiden.

3. Der Pre-Compiler

Die Aufgabe des Pre-Compilers besteht darin, aus den ID-Regeln, den LP-Regeln und den Metaregeln einer GPSG, eine (endliche) Menge von kontextfreien Regeln mit komplexen Symbolen zu erzeugen, mit denen dann der Parser arbeitet. Dieser Teil des Programms ist recht kompakt. Für den Konvertierungsprozeß werden nicht mehr als 6 LISP-Funktionen benötigt.

Wenn G eine beliebige GPSG in normaler Notation ist, dann erfolgt die Generierung der Menge der kontextfreien Regeln in zwei Schritten:

(1) Zuerst wird durch die Bildung des finiten Abschlusses der ID-Regeln unter der Menge der Metaregeln⁸ die Menge der Basis-ID-Regeln maximiert: Jede Metaregel wird auf alle die lexikalischen ID-Regeln angewendet, die nicht selbst schon das Resultat der Anwendung dieser Metaregel auf eine andere ID-Regel sind.

Allerdings führten gerade bestimmte formale Eigenschaften der Metaregeln zu dem Vorwurf, es sei prinzipiell nicht möglich, Parser für GPSGs zu entwickeln. Durch den Begriff des *finiten Abschlusses* ist zwar sichergestellt, daß die Menge der aus den Basis-ID-Regeln und Metaregeln resultierenden ID-Regeln

7 Wenn z. B. für zwei beliebige Kategorien K , K' gilt:

- (i) $K - \{s\} = K' - \{s'\}$ und
- (ii) $s = \langle m_i, w_i \rangle$ und $s' = \langle m_j, w_j \rangle$, mit $m_i \neq m_j$

dann ist zwar K mit K' kompatibel, aber keine der beiden Kategorien ist eine Extension der anderen.

8 Wenn R eine Menge von ID-Regeln ist und M eine Menge von Metaregeln, dann ist der finite Abschluß F von R unter M definiert als:

- (i) $F(R, \emptyset) = R$
- (ii) $F(R, M) = R \cup \left(\bigcup_{m \in M} f_m(R \cup F(R, M - \{m\})) \right)$,

wobei f_m einer Menge R von ID-Regeln eine Menge R' von ID-Regeln zuordnet, mit $R' = \{r' : r \in R, \text{ so daß } r' \text{ das Resultat der Anwendung der Metaregel } m \text{ auf } r \text{ ist}\}$; vgl. GKPS: 72.

endlich ist, zu einer beachtlichen numerischen Explosion können sie dennoch führen: Ausgehend von sechs Metaregeln (GKPS) und der Annahme, daß eine Metaregel aus einer ID-Regel genau eine neue ID-Regel⁹ erzeugt, ergibt sich ein

Faktor von ca. 2000^{10} d.h. z.B., aus 40 lexikalischen ID-Regeln könnte man insgesamt 80 000 ID-Regeln erhalten.

Auch wenn keine im GPSG-Format verfaßte Grammatik einer natürlichen Sprache jemals diese theoretische Obergrenze ausschöpfen dürfte, stellt sich die Frage, ob nicht eine andere Lösung möglich ist.

Im Zusammenhang mit diesem Problem lassen sich ganz verschiedene Strategien verfolgen: Z. B. könnte man

- (a) auf Metaregeln ganz verzichten und an ihrer Stelle Kategorienkookkurrenzregeln (category co-occurrence rules) und *meta-lexikalische* Regeln verwenden, wie sie in Kilbury (1986) formuliert wurden, oder
- (b) durch Angabe von Metaregel-Iterationsschemata explizit festlegen, welche Interaktion zwischen den Metaregeln stattfindet.¹¹

(2) Im nächsten Schritt wird aus der maximalen Menge von ID-Regeln ID_m unter Berücksichtigung der LP-Regeln eine Menge R von kontextfreien Regeln erzeugt. Wenn man zusätzliche Merkmalinstantiierungen zunächst ausschließt, dann gilt, daß jede ID-Regel eine Menge¹² von Strukturen (p-Strukturen) zuläßt, die sich nur durch die Links-Rechts-Beziehung der sie konstituierenden Kategorien unterscheiden. So besteht für eine ID-Regel $r = x \rightarrow y_1, y_2, y_3$ die Menge der p-Strukturen aus folgenden sechs Strukturen:

Für jede ID-Regel $r \in ID_m$ wird diese Menge von p-Strukturen gebildet, aus der dann alle Strukturen getilgt werden, die nicht LP-akzeptabel (GKPS: 99, Definition 9) sind. Jede der verbleibenden Strukturen kann als eine kontextfreie Regel aufgefaßt werden.

Wenn σ und δ Funktionen sind, so daß σ jedem $r \in ID_m$ die Menge aller p-

9 Diese Annahme ist im allgemeinen nicht erfüllt: Die meisten bislang vorgeschlagenen Metaregeln ordnen jeder ID-Regel eine Menge neuer ID-Regeln zu:

10 Es gilt: $\text{card}(F(R, M)) = r + r * \sum_{k=0}^m \prod_{i=0}^{k-1} (m - i)$, mit $r = \text{card}(R)$ und $m = \text{card}(M)$.

11 Beide Lösungen sind nicht unproblematisch. Die Verwendung meta-lexikalischer Regeln führt zu einer „klassischen“ Problemverschiebung: Genau die Probleme, die zunächst auftraten, als es galt, die Interaktion der Metaregeln in adäquater Form zu spezifizieren, stellen sich jetzt bezüglich der meta-lexikalischen Regeln. – Die Formulierung von Iterationsschemata für Metaregeln dagegen, führt dazu, daß der ohnehin schon sehr umfangreiche Formalismus der GPSG noch komplexer wird.

12 Wenn $r = \langle x, y \rangle$ eine ID-Regel ist, dann hängt die Zahl der durch sie zugelassenen p-Strukturen von der Kardinalität des Multisets y ab. Wenn $\text{card}(y) = n$ ($n \geq 1$), dann ist die Zahl der p-Strukturen kleiner oder gleich der Fakultät von n .

Strukturen zuordnet und δ einer Menge von Strukturen die einer Menge L von LP-Regeln zulässigen Strukturen zuordnet, dann gilt:

$$R = \bigcup_{r \in D_m} \delta(\sigma(r), L).$$

4. Der Parser

4.1. Die funktionale Struktur des Parsers

Das Analyseprogramm basiert auf dem als *active chart parser* bezeichneten Parsingalgorithmus für kontextfreie Grammatiken aus Winograd (1983). Die Vorteile eines active-chart-Parsers liegen darin, daß er eine große Klasse kontextfreier Grammatiken akzeptiert, daß er nicht wie mit Backtracking arbeitende Parser Strukturen mehrfach erzeugt und daß es außerdem möglich ist, verschiedene Suchstrategien zu verwenden.

In der jetzigen Implementierung arbeitet der Parser von links nach rechts und verfolgt „parallel“ alle Möglichkeiten, gefundene Teilstrukturen zu ergänzen (*breadth-first*). Er bricht erst ab, wenn er alle Strukturbeschreibungen (Resultatstrukturen) eines Satzes gefunden hat.

Betrachtet man die funktionale Struktur des Parsers, dann lassen sich die wichtigsten der LISP-Funktionen, aus denen der Parser besteht, in folgenden drei Gruppen zusammenfassen:

- (i) *Generierungsfunktionen*, die den Aufbau von Strukturbeschreibungen für Sätze steuern;
- (ii) *Instantiierungsfunktionen*, die sicherstellen, daß zum Aufbau von Strukturbeschreibungen nur lokale Bäume verwendet werden, die jedes der drei Instantiierungsprinzipien erfüllen, und
- (iii) *Aktualisierungsfunktionen*, die sicherstellen, daß Merkmalspezifikationen – wenn erforderlich – auch in tiefer eingebetteten Kategorien instantiiert werden.

Insofern läßt sich sagen, daß der Parser aus drei funktionalen Komponenten besteht, die die *Generierung* und *Aktualisierung* von Strukturen, wie die *Instantiierung* von Merkmalspezifikationen determinieren.

4.2. Datenstrukturen

Die umfassendste Datenstruktur mit der der Parser operiert, bildet die *active chart* $L = \langle V, E, P \rangle$. E, P sind zwei Folgen von *Abschnitten* und $V = \langle 1, \dots, n \rangle$ ist eine Folge von Zahlen, wobei der Wert von n von der Länge des zu verarbeitenden Satzes S abhängt. Es gilt: $n = \lg(S) + 1$.

Definition 2: Abschnitt

Ein Abschnitt $A = \langle i, H, j, R \rangle$ besteht aus

- (i) zwei Zahlen i, j , mit $1 \leq i \leq j \leq n$;
- (ii) einer möglicherweise leeren Folge von Kategorien $R = \langle k_1, \dots, k_m \rangle$ ($m \geq 0$);
- (iii) dem komplexen Ausdruck $H = \langle \langle X \ Y_1 \ \dots \ Y_n \rangle \langle Y^1 \rangle \dots \langle Y^n \rangle \rangle$, ($n \geq 1$). Ein komplexer Ausdruck ist durch folgende Eigenschaften charakterisiert:
 - a) $X, Y_1, \dots, Y_n$ sind beliebige Kategorien, und X wird *Leitkategorie* genannt.
 - b) Für alle Y^i ($1 \leq i \leq n$) von H gilt:
 $Y^i = \langle Y'_i, w \rangle$, Y'_i ist eine lexikalische Kategorie und w ein Wort dieser Kategorie, oder
 Y^i ist ein komplexer Ausdruck mit Y'_i als Leitkategorie.
 - c) Für alle Y_i, Y'_i gilt: Y_i und Y'_i sind zwei miteinander kompatible Kategorien.

4.3. Arbeitsweise des Parsers

Es gibt zwei Typen von Abschnitten: Wenn $A = \langle i, H, j, R \rangle$ ein beliebiger Abschnitt ist, dann wird A ein *vollständiger Abschnitt* genannt gdw. $R = \emptyset$; sonst ist A ein *offener Abschnitt*.

Vereinfacht läßt sich sagen, daß die Abschnitte aus E die vom Parser gefundenen Teilstrukturen repräsentieren und die Abschnitte aus P mögliche Ergänzungen dieser Strukturen darstellen.

Die Arbeitsweise des Parsers ist durch folgende drei Operationen¹³ geprägt:

(a) Initialisierung von L

R sei die Menge der Regeln, die der Pre-Compiler für eine beliebige GPSG G erzeugt hat, D ein Lexikon und $S = w_1 \dots w_n$ ($n \geq 1$) ein beliebiger Satz. Für $L = \langle V, E, P \rangle$ gilt:

- (i) $V = \langle 1, \dots, n+1 \rangle$.
- (ii) E ist eine Folge von vollständigen Abschnitten der Form $\langle i, \langle \langle K_D, w_i \rangle \rangle, i+1, \emptyset \rangle$, mit K_D als einer lexikalischen Kategorie. Für jede lexikalische Kategorie aus dem Lexikoneintrag von w_i gibt es genau einen Abschnitt in E .
- (iii) P ist eine Folge von offenen Abschnitten. Für jede Regel $S \rightarrow x_1 \dots x_m$ ¹⁴

¹³ Diese Darstellung ist relativ knapp gehalten. Eine ausführlichere Beschreibung des dem Parser zugrundeliegenden Algorithmus findet sich in Winograd (1983: 116–127).

¹⁴ Wegen der besseren Lesbarkeit, verwende ich hier zur Bezeichnung von Kategorien keine komplexen Symbole.

($m \geq 1$) $\in R$ gibt es in P einen Abschnitt der Form $\langle 1, \langle \langle S \rangle \rangle, 1, \langle x_1 \dots x_m \rangle \rangle$.

(b) *Kombinieren von Abschnitten*

Wenn a, a' zwei beliebige Abschnitte sind, dann kann a mit a' kombiniert werden gdw.

- (i) $a = \langle b, H, e, R \rangle$, mit $e \geq b$, $H = \langle \langle X Y_1 \dots Y_i \rangle \langle Y^1 \rangle \dots \langle Y^i \rangle \rangle$ ($i \geq 0$) und $R = \langle Y_{i+1} \dots Y_n \rangle$ ($n \geq 1$);
- (ii) $a' = \langle e, H', f, R' \rangle$, mit $f > e$, $R' = \emptyset$ und $H' = \langle \langle Y_{i+1}' Z_1 \dots Z_m \rangle \langle Z^1 \rangle \dots \langle Z^m \rangle \rangle$ und Y_{i+1}, Y_{i+1}' sind zwei kompatible Kategorien.

Das Ergebnis ist ein neuer Abschnitt $a'' = \langle b, H'', f, R'' \rangle$, mit $H'' = \langle \langle X Y_1 \dots Y_{i+1}^* \rangle \langle Y^1 \rangle \dots \langle Y^{i+1} \rangle \rangle$, $R'' = \langle Y_{i+2} \dots Y_n \rangle$ ($n \geq 0$), $Y^{i+1} = \langle \langle Y_{i+1}' Z_1 \dots Z_m \rangle \langle Z^1 \rangle \dots \langle Z^m \rangle \rangle$ und $Y_{i+1}^*, Y_{i+1}'^*$ sind die Kategorien, die man aus Y_{i+1}/Y_{i+1}' erhält, indem man in ihnen alle Merkmalspezifikationen aus Y_{i+1}'/Y_{i+1} instantiiert, die nicht schon in ihnen enthalten sind.

(c) *Bildung neuer Abschnitte*

Wenn $a = \langle i, H, j, R \rangle$ ein offener Abschnitt ist, mit $R = \langle K_1, \dots, K_n \rangle$ ($n \geq 1$), dann wird für jede Regel $K_1' \rightarrow y_1 \dots y_m \in R$ ($m \geq 1$), für die gilt: K_1' ist eine mit K_1 kompatible Kategorie, ein neuer Abschnitt $\langle j, \langle \langle K_1' \rangle \rangle, j, \langle y_1 \dots y_m \rangle \rangle$ gebildet.

4.4. Tilgungsregeln und Ableitungen

Die Behandlung von Tilgungsregeln ist in einem active-chart-Parser relativ unproblematisch. Die Strukturbeschreibungen werden *bottom-up* aufgebaut, indem ein offener mit einem geeigneten vollständigen Abschnitt kombiniert wird.

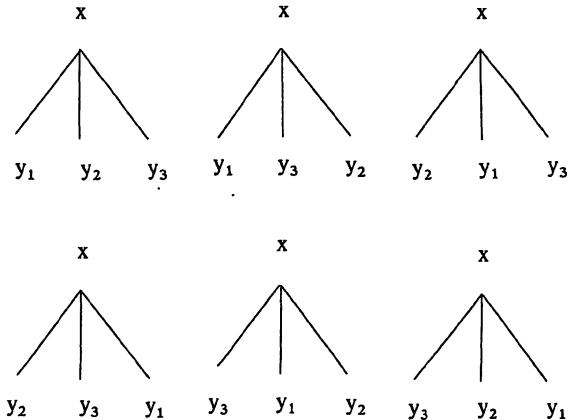
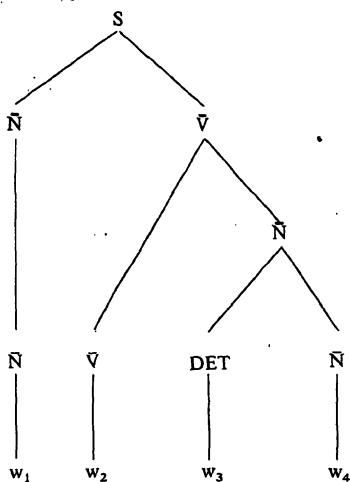
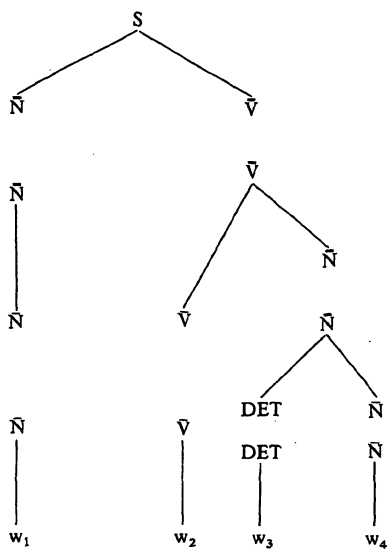


Fig. 3



(a)



(b)

Fig. 4

Jeder auf diese Weise neugebildete Abschnitt muß daraufhin überprüft werden, ob seine Reststruktur mit einer oder mehreren Kategorien beginnt, die die Merkmalspezifikation $\langle \text{NULL} + \rangle$ enthalten. Diese Kategorien sind dann aus der Reststruktur zu tilgen und in die Kopfstruktur zu übertragen:

$$\langle i \langle X \langle Y_1 \dots Y_i \rangle \langle Y^1 \rangle \dots \langle Y^i \rangle \rangle j \langle Y_{m\langle \text{NULL} + \rangle} Y_{m+1} \dots Y_n \rangle \rangle$$

$$\langle i \langle X \langle Y_1 \dots Y_i Y_{m\langle \text{NULL} + \rangle} \rangle \langle Y^1 \rangle \dots \langle Y^i \rangle \rangle j \langle Y_{m+1} \dots Y_n \rangle \rangle$$

Wenn $S = w_1 \dots w_n$ ($n \geq 1$) ein beliebiger Satz ist, dann wird aus jedem Abschnitt a der Form $\langle 1, H, n+1, R \rangle$, mit $R = \emptyset$ und $H = \langle \langle S Y_1 \dots Y_m \rangle \langle Y_1 \rangle \dots \langle Y_m \rangle \rangle$ ($m \geq 1$) eine Resultatstruktur für S gebildet und ausgegeben.

Der Parser bildet für einen Satz zunächst keine Strukturbeschreibung des üblichen Formats, sondern verwaltet die strukturellen Informationen als eine Folge von „lokalen Bäumen“. Wenn $S = w_1 w_2 w_3 w_4$ ein Satz mit der in Fig. 4 (a) dargestellten Struktur ist, dann erzeugt der Parser zunächst einen Abschnitt der Form:

$$\langle 1 \langle \langle S \bar{N} \bar{V} \rangle \langle \langle \bar{N} \bar{N} \rangle \langle \langle \bar{N} w_1 \rangle \rangle \rangle \langle \langle \bar{V} \bar{V} \bar{N} \rangle \langle \langle \bar{V} w_2 \rangle \rangle \rangle \langle \langle \bar{N} \text{DET} \bar{N} \rangle \langle \langle \text{DET} w_3 \rangle \rangle \rangle \langle \langle \bar{N} w_4 \rangle \rangle \rangle \rangle 5 \emptyset \rangle \text{ (s. Fig. 4).}$$

Die Tilgung der redundanten Kategorien erfolgt erst bei der Ausgabe der Struktur.

Eine weitere Besonderheit des Parsers liegt darin, daß Kategorien nicht einfach als Mengen von Merkmalspezifikationen behandelt werden, sondern jede Kategorie $K = \langle h, t \rangle$ als ein geordnetes Paar repräsentiert wird, das aus der Menge h der inhärenten Merkmalspezifikationen und der Menge t der instantiierten Merkmalspezifikationen besteht. Während die Menge der inhärenten Merkmalspezifikationen konstant bleibt¹⁵, kann prinzipiell jeder Ableitungsschritt zu einer Änderung von t führen.

4.5. Instantiierungsprinzipien

Die drei Instantiierungsprinzipien werden direkt in den Ableitungsprozeß eingebunden: Sobald durch Kombination zweier Abschnitte ein neuer vollständiger Abschnitt $a = \langle i, H, j, \emptyset \rangle$ entsteht, werden auf den komplexen Ausdruck

15 Eine Ausnahme ist allerdings zu beachten: Wenn m_i, m_j zwei Merkmalspezifikationen sind und $K = \langle h, t \rangle$ eine beliebige Kategorie ist, so daß gilt:

- (i) $m_i = \langle \alpha, \beta \rangle$, mit $\alpha \notin \text{Atom}$;
- (ii) $m_i \in h$ und
- (iii) $m_j \notin \beta$,

dann wird in der Kategorie K' , die man aus K erhält, indem man m_j in β instantiiert, m_j weiterhin als inhärente Merkmalspezifikation aufgefaßt und m_j nicht anders behandelt, als die anderen Merkmalspezifikationen von β .

$H = \langle \langle X Y_1 \dots Y_m \rangle \langle Y^1 \rangle \dots \langle Y^m \rangle \rangle$ ($m \geq 1$) das Foot-Feature-Prinzip, das Control-Agreement-Prinzip und die Head-Feature-Convention (in dieser Reihenfolge) angewendet, und zwar zunächst auf den lokalen Baum $\langle X Y_1 \dots Y_m \rangle$ und dann, soweit erforderlich, auf die lokalen Bäume, die die Teilstrukturen $\langle Y^1 \rangle \dots \langle Y^m \rangle$ konstituieren.

4.5.1. Das Foot-Feature-Prinzip

Das Foot-Feature-Prinzip (FFP) fordert, daß in einem lokalen Baum $\langle X Y_1 \dots Y_m \rangle$ die Menge der in X instantiierten Foot-Merkmale mit der Unifikation der instantiierten Foot-Merkmale aller Tochterkategorien identisch ist (GKPS: 81 ff.)

Die Distribution von Foot-Merkmalen läßt sich als ein rein „bottom-up“ arbeitender Prozeß beschreiben, der die Instantiierung von Foot-Merkmalen in Tochterkategorien von bestimmten Bedingungen abhängig macht, deren Erfüllung garantiert, daß alle in ihnen instantiierten Foot-Merkmale auch in X instantiiert werden können.

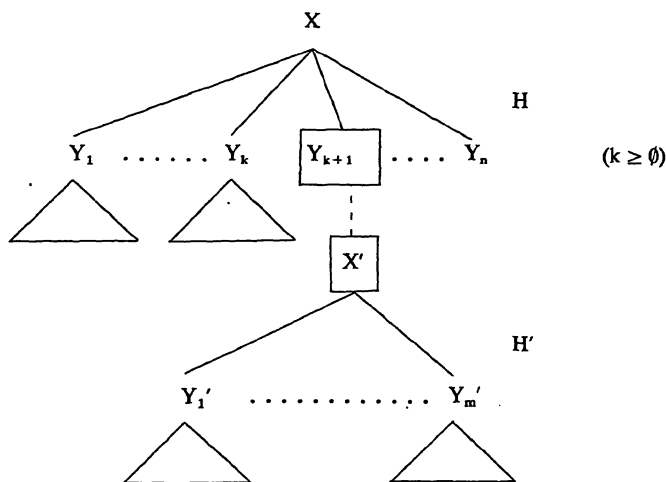


Fig. 5

Wenn $a = \langle i, H, j, \langle Y_{k+1} \dots Y_n \rangle \rangle$ ($n \geq 1$) und $a' = \langle j, \langle \langle X' Y_1' \dots Y_m' \rangle \dots \rangle, k, \emptyset \rangle$ zwei beliebige miteinander kombinierbare Abschnitte sind, die die Strukturen H bzw. H' festlegen (s. Fig. 5) und Y_{k+1}^*, X'^* die Kategorien sind, die man aus Y_{k+1}/X' erhält, indem man in ihnen alle Merkmalspezifikationen aus X'/Y_{k+1} instantiiert, die nicht schon in ihnen enthalten sind, dann kann der aus

ihrer Kombination resultierende Abschnitt a'' nur dann das FFP erfüllen, wenn folgende vier Bedingungen erfüllt sind:

- (i) Y_{k+1} und X' sind zwei miteinander kompatible Kategorien;
- (ii) es gibt kein Foot-Merkmal, das in Y_{k+1} , nicht aber in X' spezifiziert ist, d. h.,
 $\forall f (f \in \text{DOM}(Y_{k+1}|\text{FOOT}) \rightarrow f \in \text{DOM}(X'|\text{FOOT}))^{16}$;
- (iii) für jedes Foot-Merkmal f , mit $f \in Y_{k+1}^*$ und $f \notin Y_{k+1}$, gilt:
 - a) es gibt keine Kategorie Y_i ($1 \leq i \leq k$), in der f instantiiert wurde, oder
 - b) wenn es eine Kategorie Y_i gibt, in der f instantiiert wurde, dann ist $Y_i(f) = Y_{k+1}^*(f)$;
- (iv) es gibt kein Foot-Merkmal f , mit $f \in Y_{k+1}^*$, $f \notin Y_{k+1}$ und $f \in \text{DOM}(X)$, d. h., f gehört nicht zu den inhärenten Foot-Merkmalen von X ¹⁷.

Wenn diese vier Bedingungen beachtet werden, dann ist leicht sicherzustellen, daß nur Abschnitte gebildet werden, deren komplexer Ausdruck ausschließlich aus solchen lokalen Bäumen besteht, die das FFP erfüllen.

Jeder neugebildete vollständige Abschnitt $\alpha = \langle i, H, j, \emptyset \rangle$, mit $H = \langle \langle X Y_1 \dots Y_n \rangle \dots \rangle$, wird ersetzt durch einen Abschnitt $\alpha' = \langle i, \langle \langle X' Y_1 \dots Y_n \rangle \dots \rangle j, \emptyset \rangle$, wobei man X' aus X erhält, indem man in X die Unifikation aller in den Tochterkategorien instantiierten Foot-Merkmale instantiiert.

Da alle Foot-Merkmale kategorienwertige Merkmale sind, kann es allerdings unter bestimmten Voraussetzungen notwendig sein, Merkmalspezifikationen in einigen Foot-Merkmalen schon gebildeter Strukturen zu instantiieren. Wenn z. B. gilt:

- (i) $f \in \text{DOM}(Y_{k+1}|\text{FOOT})$;
- (ii) $f \in \text{DOM}(X'|\text{FOOT})$;
- (iii) m_i ist eine Merkmalspezifikation, mit $m_i \in Y_{k+1}(f)$ und $m_i \notin X'(f)$

und f außerdem zu den instantiierten und nicht zu den inhärenten Merkmalen von X' gehört, dann ist m_i nicht nur in X' , sondern auch in den Tochterkategorien von X' zu instantiieren, die für die Instantiierung von f in X' verantwortlich sind, sowie in den Nachstrukturen dieser Kategorien. Die Aktualisierung von Strukturen übernimmt im Parser eine spezielle Funktion.

Das FFP nimmt nur Einfluß auf die in den Kategorien eines lokalen Baumes instantiierten Foot-Merkmale. In GKPS wird SLASH nicht nur durch Regeln explizit eingeführt, sondern in bestimmten Fällen auch durch freie Instantiierung. So sorgt z. B. die SLASH TERMINATION METARULE 1 (STM1) für die Generierung von Regeln der Form ' $X \rightarrow W, X^2[\text{NULL} +]$ ' (GKPS: 143). Die FCR 19 erzwingt die Instantiierung von SLASH in der mit $[\text{NULL} +]$ spezifi-

16 Für ein Merkmal f und eine Kategorie $Y = \langle h, t \rangle$ gilt: $f \in \text{DOM}(Y)$ gdw. es eine Merkmalspezifikation $m = \langle f, w \rangle$ gibt, mit $m \in h$ oder $m \in t$.

17 Zu diesem Zeitpunkt kann X keine instantiierten Merkmale enthalten.

zierten Kategorie. Da SLASH in diesem Fall zu den instantiierten Foot-Merkmalen der Kategorie gehört, garantiert das FFP, daß die Kategorie alle Informationen über die getilgte Konstituente enthält (GKPS: 92 f.).

Da der Parser keine freie Instantiierung zuläßt, gibt es in diesen Fällen Probleme:

- (i) Es muß eine Möglichkeit gefunden werden, auch jene lokalen Bäume zu erhalten, welche durch die mit STM1 erzeugten Regeln zugelassen werden, und
- (ii) es muß sichergestellt werden, daß diese SLASH-Kategorien die notwendigen Merkmalspezifikationen enthalten.

Das erste Problem läßt sich lösen, indem durch STM1 nicht nur die Merkmalspezifikation $\langle \text{NULL} + \rangle$, sondern gleichzeitig auch die durch die FCR geforderte SLASH-Kategorie eingeführt wird. Allerdings gehört dann diese SLASH-Kategorie zu den inhärenten Merkmalen der Kategorie und unterliegt daher nicht dem FFP.

Es wäre leicht, dieses Problem durch die Aktualisierungsfunktionen zu lösen. Da ich es aber für ein fragwürdiges Vorgehen halte, wenn man theoretische Probleme dadurch zu lösen versucht, indem man das Programm ad hoc um spezielle Prozeduren erweitert, habe ich mich für eine andere Lösung entschieden. Der Parser arbeitet mit einer Variante des FFPs, die fordert, daß auch die inhärenten Foot-Merkmale einer bestimmten Bedingung genügen müssen:

Definition 3: *Das erweiterte Foot-Feature Prinzip (FFP⁺)*

Wenn $\alpha = \langle X Y_1 \dots Y_n \rangle$ ($n \geq 1$) ein beliebiger lokaler Baum ist, dann erfüllt α das erweiterte Foot-Feature Prinzip gdw.

- (i) $X|\text{FOOT}_{\text{inst}} = \bigcup_{1 \leq i \leq n} Y_i|\text{FOOT}_{\text{inst}}$ ¹⁸ und
- (ii) $\forall m ((m \in X|\text{FOOT}_{\text{inh}}) \rightarrow \exists Y_i (m \in Y_i|\text{FOOT}_{\text{inh}}))$ ¹⁹.

Für diese restriktivere Fassung des FFP gibt es aus linguistischer Sicht einen guten Grund:

Das FFP läßt, anders als FFP⁺, Strukturen zu, in denen es Kategorien mit nicht-terminierten Foot-Merkmalen gibt. Dieser Fall tritt immer dann ein, wenn es in einem lokalen Baum α in der Mutterkategorie X ein inhärentes Foot-Merkmal m gibt, das in keiner der Töchter von X vorkommt. So kann es zu Strukturen kommen die SLASH-Kategorien enthalten, aber keine „Lücken“, in denen es nicht-lexikalische Kategorien gibt, die für WH oder RE spezifiziert sind, aber keine der lexikalischen Kategorien für diese Merkmale spezifiziert sind²⁰.

18 Diese Bedingung entspricht der alten Fassung des FFP.

19 $Y|\text{FOOT}_{\text{inst}}$ bzw. $Y|\text{FOOT}_{\text{inh}}$ bezeichnet die Menge der instantiierten bzw. inhärenten Foot-Merkmale einer Kategorie Y.

20 Die Foot-Merkmale WH und RE werden in der GPSG zur Markierung bestimmter Typen von Pronomina (Frage-, Relativ- und Reflexivpronomina) verwendet.

So besteht z. B. der Baum in Fig. 6, den die beiden Regeln $X \rightarrow Y_1 Y_{2\langle WH \rangle}$ und $Y_{2\langle WH \rangle} \rightarrow Z_1 Z_2$ festlegen, aus zwei lokalen Bäumen, die beide das FFP erfüllen.

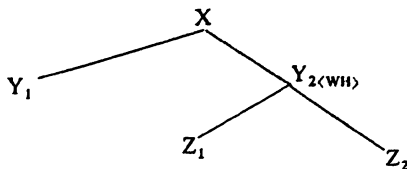


Fig. 6

Solche unerwünschten Resultate schließt FFP⁺ aus, indem es fordert, daß es für jedes in der Mutterkategorie X inhärente Foot-Merkmal m mindestens eine Tochter gibt, zu deren inhärenten Merkmalen auch m gehört.

Um eine korrekte Behandlung von SLASH-Kategorien zu garantieren, sind die Metaregeln STM1 und STM2 zu ersetzen durch:

- (STM1*): $X \rightarrow W, X^2$
 $\quad \quad \quad \downarrow$
 $X/X^{2'} \rightarrow W, X^2[+ \text{NULL}]/X^2$
 (STM2*): $X \rightarrow W, V^2[+ \text{SUBJ}, \text{FIN}]$
 $\quad \quad \quad \downarrow$
 $X/\text{NP} \rightarrow W, X[+ \text{NULL}]/\text{NP}, V^2[-\text{SUBJ}].$

4.5.2. Das Control-Agreement-Prinzip

Anders als beim FFP beschränkt sich die durch das Control-Agreement-Prinzip (CAP) induzierte Distribution von Merkmalen ausschließlich auf lokale Bäume. Wenn $\alpha = \langle X Y_1 \dots Y_n \rangle$ ($n \geq 1$) ein beliebiger lokaler Baum ist, dann unterscheidet CAP zwei Typen von Kontrollbeziehungen:

- (i) Es kann eine Kontrollbeziehung zwischen zwei Tochterkategorien Y_i, Y_j bestehen, und
- (ii) es kann eine Kontrollbeziehung zwischen X und einer prädikativen Kategorie Y_i geben.

Um Kontrollbeziehungen für den Parser transparent zu gestalten, wird ein „dummy“-Merkmal C mit den Werten $\{+, -\}$ verwendet, wobei die Merkmalspezifikationen $\langle C + \rangle$ und $\langle C - \rangle$ eine Kategorie als „controller“ bzw. als „controlee“ markieren. Es wird vorausgesetzt, daß bereits die Kategorien der ID-Regeln entsprechend markiert sind²¹. Unter dieser Voraussetzung erfolgt dann der Transfer von Merkmalen so, wie er vom CAP gefordert wird.

21 In der jetzigen Fassung werden nur ID-Regeln akzeptiert, in denen eine Kontrollbeziehung ausgezeichnet ist. Eine Anpassung an ID-Regeln mit mehreren Kontrollbeziehungen oder Kontrollbeziehungen zwischen einem „controller“ und mehreren „controlees“ ist aber durchaus zu realisieren.

Ich halte diese, man könnte sagen „reduktionistische“ Behandlung von CAP für akzeptabel, da

- a) für beide Typen von Kontrollbeziehungen die *semantischen Typen* der beteiligten Kategorien entscheidend sind und, meiner Auffassung nach, ein Parser möglichst vermeiden sollte, direkt auf semantische Informationen zurückzugreifen, und
- b) die Instantiierung von Merkmalspezifikationen keinen Einfluß auf Kontrollbeziehungen haben soll²² und so der Festlegung von Kontrollbeziehungen über den Kategorien von ID-Regeln nichts im Wege steht.

4.5.3. Die Head-Feature-Konvention

Folgt man GKPS, dann erfüllt ein lokaler Baum $\alpha = \langle X Y_1 \dots Y_n \rangle$ nur dann die Head-Feature-Konvention (HFK), wenn

- (a) für jede Head-Kategorie Y_i von α gilt, daß $Y_i|HEAD$ eine Extension der Kategorie ist, die die Menge der Head-Merkmale bilden, die in Y_i frei instantiierbar sind;
- (b) $X|HEAD$ eine Extension der Kategorie ist, die aus allen in X frei instantiierbaren Head-Merkmalen besteht, die in allen Head-Kategorien von α enthalten sind, und
- (c) für jede Head-Kategorie Y_i von α gilt: BAR ist ein inhärentes Merkmal von Y_i (s. GKPS: 95 und 97).

Da nur Regeln mit genau einer Head-Kategorie akzeptiert werden, konnte eine sehr einfache Implementierung für die Head-Feature-Konvention gewählt werden, die aber die der HFK zugrundeliegende Idee berücksichtigt: „The job of the HFC will be to determine the feature composition of the head category or categories thus identified, essentially by requiring the set of **HEAD** features that get instantiated on heads to be identical to (or a superset of) the **HEAD** features instantiated on the mother category“ (GKPS: 51).

Ähnlich wie beim FFP, kann der die Distribution der Head-Merkmale steuernde Prozeß als ein im wesentlichen „bottom-up“ arbeitender Prozeß verstanden werden²³: Zunächst werden über das Lexikon Head-Merkmale in den Head-Kategorien eines lokalen Baums instantiiert, und anschließend wird die Schnittmenge dieser Merkmale in der Mutterkategorie des lokalen Baums instantiiert.

22 vgl. GKPS: 87: „[...] we must modify the definition of control in such a way that it ignores perturbations of semantic type occasioned by the presence of instantiated FOOT features.“

23 Eine Ausnahme gibt es: Wenn z. B. in Fig. 3 die Kategorie Y_{k+1} Head-Merkmale enthält, die in X' zu instantiieren sind, dann sind diese Merkmale auch in den Head-Kategorien von $\langle X' Y_1' \dots Y_m' \rangle$ zu instantiieren.

Head-Kategorien werden in den ID-Regeln durch die Merkmalspezifikation $\langle H + \rangle$ markiert, wobei H wie C ein „dummy“ Merkmal ist, dessen einzige Funktion darin besteht, Kategorien zu markieren, um bestimmte Aktionen des Parsers auszulösen.

5. Der Post-Filter

Die beiden Regeltypen, auf die ich bislang nicht eingegangen bin, die FSDs und FCRs, werden nicht im Zusammenhang mit dem Generierungsprozeß berücksichtigt. Sie werden in einem gesonderten Programmteil verarbeitet, das die Menge der vom Parser für einen akzeptierten Satz erzeugten Strukturen als Input nimmt und eine möglicherweise leere Menge von Resultatstrukturen liefert.

Wenn G eine beliebige GPSG ist und α eine Struktur für einen beliebigen Satz S ist, dann ist α eine *Resultatstruktur* von S gdw. α vollständig und akzeptabel ist.

Eine Struktur α ist *vollständig* relativ zu einer GPSG G gdw. alle Kategorien in α alle FSDs aus G erfüllen.

Eine Struktur α ist *akzeptabel* relativ zu einer GPSG G gdw. alle Kategorien in α alle FCRs aus G erfüllen; d. h. wenn α ausschließlich *legale* Kategorien enthält.

Der Post-Filter verfährt in zwei Schritten. Zuerst erzeugt er aus den Strukturen, die der Parser einem Satz zuordnet, vollständige Strukturen, die er dann auf ihre Akzeptabilität überprüft. Zwei Punkte gilt es allerdings zu beachten:

Anders als in GKPS werden nur die FCRs, nicht aber die FSDs als Boolesche Funktionen aufgefaßt. Die FSDs haben hier die Aufgabe, unterspezifizierte Kategorien um bestimmte Merkmalspezifikationen zu erweitern, d. h., es sind Funktionen über der Menge der Kategorien von G. Wenn $f = \langle \alpha, \beta \rangle$ ein FSD von G ist mit $\beta = \{b_1, \dots, b_n\}$ als einer Menge von Merkmalspezifikationen und K eine Kategorie von G mit $f \sqsubset_{\text{EX}} K$, dann gilt:

$$f(K) = K'$$

mit K' als dem Ergebnis der *bedingten Instantiierung* von α in K.

Definition 4: Bedingte Instantiierung

Wenn k_1, k_2 zwei beliebige Kategorien sind, dann ist die Kategorie K das Ergebnis der bedingten Instantiierung von k_2 in k_1 ($K = I(k_1, k_2)$) gdw.

- (i) $k_1 \sqsubset_{\text{EX}} K$;
- (ii) $\forall(m) ((m \in \text{Atom} \wedge m \in \text{DOM}(K) - \text{DOM}(k_1)) \rightarrow K(m) = k_2(m))$;
- (iii) $\forall(m) (m \notin \text{Atom} \rightarrow (K(m) = I(k_1(m), k_2(m))))$.

Charakteristisch für die bedingte Instantiierung von Merkmalspezifikationen ist, daß die inhärenten Merkmalspezifikationen der Kategorie Vorrang gegen-

über den zu instantiierenden Merkmalspezifikationen besitzen. So ist eine adäquate Behandlung der FSDs gewährleistet.

Der zweite Punkt betrifft das systematische Verhältnis zwischen den FSDs und den FCRs: Der Post-Filter basiert auf der Annahme, daß FSDs und FCRs voneinander *logisch unabhängig* sind:

Definition 5: Logische Unabhängigkeit

Wenn G eine beliebige GPSG ist und F_s bzw. F_c die Menge der FSDs/FCRs von G sind, dann sind F_s und F_c *logisch unabhängig* gdw. für jede legale Kategorie K aus G gilt, daß auch die durch K und F_s determinierte Kategorie K' akzeptabel ist.

Anders formuliert: Die Anwendung von FSDs auf eine Kategorie darf nichts an deren Status, d. h. der *Legalität* der Kategorie ändern.²⁴

6. Resümee

Die wichtigsten Eigenschaften des beschriebenen Parsers lassen sich in sechs Punkten zusammenfassen:

1. ID-Regeln, LP-Regeln und Metaregeln werden als eine komplexe Notationsvariante für kontextfreie Syntaxen (mit komplexen Symbolen) betrachtet.
2. Es gibt keine „freie Instantiierung“ von Merkmalen/Merkmalspezifikationen. Unterspezifizierte Kategorien werden, wenn man nur einmal den wichtigsten Fall betrachtet, um die zusätzlichen Merkmalspezifikationen erweitert, die das Lexikon liefert: Sie werden zunächst in den lexikalischen Kategorien instantiiert und dann, durch die drei Instantiierungsprinzipien, über die Kategorien der Struktur verteilt.
3. Zur Markierung solcher Kategorien, die in lokalen Bäumen eine spezielle Funktion ausüben („controller“, „controlee“ und Head-Kategorien), werden spezielle „dummy“-Merkmale verwendet.
4. Die FSDs und FCRs werden nicht in den Generierungsprozeß eingebunden, sondern als ein Filter über Resultatstrukturen formuliert.
5. FSDs und FCRs sind voneinander logisch unabhängig.
6. Die FSDs werden nicht als Boolesche Funktionen aufgefaßt, sondern als Funktionen, die unterspezifizierte Kategorien um neue Merkmalspezifikationen erweitern.

²⁴ Wie man leicht feststellen kann, erfüllt das Syntaxfragment in GKPS diese Forderung.

Literaturnachweis

- [Gazdar/Pullum 1982] Gazdar, Gerald/Pullum, Geoffrey K.: *Generalized Phrase Structure Grammar: a theoretical synopsis*. – Bloomington, Indiana: Indiana University Linguistics Club 1982. [Vervielf.].
- [GKPS] Gazdar, Gerald/Klein, Ewan/Pullum, Geoffrey/Sag, Ivan: *Generalized Phrase Structure Grammar*. – (Oxford): Basil Blackwell (1985).
- [Kilbury 1986] Kilbury, James: *Category cooccurrence restrictions and the elimination of metarules*. – In: 11th International Conference on Computational Linguistics. Proceedings of Coling '86. Bonn, August, 25th to 29, 1986. – Bonn: University of Bonn 1986. S. 50–55.
- [Shieber 1984] Shieber, Stuart M.: *Direct parsing of ID/LP grammars*. – In: *Linguistics and Philosophy* 7 (1984), 135–154.
- [Shieber 1986] –: *A simple reconstruction of GPSG*. – In: 11th International Conference on Computational Linguistics. Proceedings of Coling '86. Bonn, August, 25th to 29, 1986. – Bonn: University of Bonn 1986. S. 211–215.
- [Winograd 1983] Winograd, Terry: *Language as a cognitive process*. Volume I: *Syntax*. – Reading, Massachusetts [etc.]: Addison-Wesley (1983). Eingereicht am 3. 11. 1986.

Eingereicht am 3.11.1987